# Бот выдачи кинооборудования: развёртывание и сопровождение

Инструкция для студентов и сопровождающих. Версия 1.1, 9 ноября 2026 года.

Проект: https://github.com/alexander1221/equipment_bot_cinema

Целевая система: Ubuntu Server 24.04 LTS, Linux x86_64, Docker Compose, nginx, SQLite. Руководство рассчитано на выполнение под root в консоли удалённого сервера. Для другой учётной записи системные операции требуют согласованного sudo-доступа. Секретных значений в комплекте нет.

# 1. Как пользоваться инструкцией

**Для существующего сервера не повторяйте установку с начала.** Прочитайте паспорт, проверьте состояние и используйте раздел 13. Не перезаписывайте .env, базу, сертификат или настройки nginx. На существующем сервере контейнер намеренно остановлен владельцем; для включения достаточно start, если после передачи ничего не менялось.

Для нового сервера выполняйте разделы 4–10 по порядку. При необходимости замените IP во всех сетевых командах, сертификате и nginx. Каждый блок выполняйте отдельно. Если команда завершилась ошибкой, сначала исправьте её, затем продолжайте.

Содержание:

1. Как пользоваться инструкцией 2. Паспорт текущего развёртывания 3. Устройство системы и ограничения 4. Доступ, требования и проверка сети 5. Установка Docker 6. Получение кода 7. Параметры и секреты 8. Запуск панели без Telegram 9. HTTPS по IP через nginx 10. Внешний доступ и файрвол 11. Полный режим и Telegram-прокси 12. Первичная настройка пользователей 13. Остановка, запуск, перезагрузка и применение настроек 14. Резервное копирование 15. Автоматические копии 16. Восстановление базы 17. Обновление и откат 18. Домен и сертификаты 19. Диагностика неисправностей 20. Приёмка, передача и шпаргалка

# 2. Паспорт текущего развёртывания

Состояние зафиксировано по переписке и проверкам владельца на 09.11.2026; перед изменениями проверяйте фактическое состояние.

| Параметр | Значение |
|---|---|
| Сервер | 185.11.247.226 |
| ОС и SSH | Ubuntu 24.04.1 LTS; root; TCP 22 |
| Репозиторий и ветка | alexander1221/equipment_bot_cinema, main |
| Версия кода с комментариями преподавателя | 647ede50a04248c64c1b48a1e891f83a7b0309f2 |
| Каталог приложения | /opt/equipment_bot_cinema |
| Дополнительный Compose-файл | /opt/equipment_bot_cinema.web.yml |
| Режим приложения | python main.py web — без Telegram |
| Compose | пакет Ubuntu; сообщённая версия 2.40.3 |
| Внутренний адрес | http://127.0.0.1:8080 |
| Внешний адрес | https://185.11.247.226 |
| SQLite на сервере | /opt/equipment_bot_cinema/data/equipment.db |
| SQLite в контейнере | /srv/data/equipment.db |
| Файл настроек | /opt/equipment_bot_cinema/.env |
| Сайт nginx | /etc/nginx/sites-available/equipment-bot |
| TLS | /etc/nginx/ssl/equipment-bot/server.crt и server.key |

Подтверждено: образ собран; локальный HTTP /login вернул 200; локальный HTTPS с проверкой сертификата вернул 200; владелец открыл панель с компьютера и вошёл. Затем владелец остановил контейнер через stop. nginx работает отдельно; при остановленном приложении 502 от nginx ожидаем.

Сертификат временный, самоподписанный, выпущен на 90 дней. Точную дату окончания смотрите через openssl, а не вычисляйте по дате документа. Предупреждение браузера об издателе

ожидаемо; отпечаток нужно проверять.

Telegram с сервера по IPv4 не отвечал на TCP 443, тогда как GitHub и PyPI отвечали. Это наблюдение соединения; конкретная причина не установлена. Администраторам требуется согласовать доступ или прокси. Реальный бот на сервере пока не проверен.

После изменения комментариев преподавателя в среде разработки прошли 84 теста. Это не заменяет функциональные проверки сервера. Резервное копирование, расписание, восстановление, обновление, домен и полный режим ниже описаны для последующего выполнения: их настройка на целевом сервере в переписке ещё не подтверждалась.

# 3. Устройство системы и ограничения

Путь запроса: браузер → HTTPS:443 → nginx → HTTP:127.0.0.1:8080 → Docker-контейнер → SQLite.

Образ Docker содержит код и Python-зависимости. Контейнер — работающий экземпляр образа. Compose описывает его параметры. Каталог data подключён с сервера в /srv/data; данные переживают замену контейнера. Образ и контейнер сами по себе не являются резервной копией базы.

Основной docker-compose.yml запускает полный режим и публикует 8080 на всех интерфейсах. В этом развёртывании **всегда используется дополнительный файл**, заменяющий команду и список портов. Публикация 8080 только на 127.0.0.1 позволяет nginx быть единой внешней точкой входа.

В полном режиме бот и панель работают в одном процессе и с общей базой. Бот использует исходящий HTTPS к Telegram Bot API и long polling. Входящие подключения Telegram и webhook для этой реализации не нужны. Один токен должен обслуживать только один polling-процесс.

В режиме web бот не запускается: Telegram-уведомления не отправляются. Фоновый цикл напоминаний и автоматического истечения ожидающих заявок тоже запускается только с ботом. Поэтому web-only — самостоятельная панель, но не полная замена работающему боту.

Незавершённые диалоги хранятся в памяти и сбрасываются при перезапуске процесса; пользователь начинает их заново. Уже отправленные заявки, инвентарь и регистрации находятся в SQLite и сохраняются.

# 4. Доступ, требования и сеть

Потребуются: разрешение владельца сопровождать сервер; SSH или консоль хостинга; доступ к репозиторию; возможность управлять входящими правилами; для полного режима — токен BotFather и доступ к Telegram либо согласованный прокси.

Текущему небольшому приложению достаточно Linux x86_64 с 1 ГБ RAM, swap и несколькими гигабайтами свободного диска. На сервере сообщалось около 961 МиБ RAM, swap 1 ГиБ и 15 ГиБ свободного диска. На сборке следите за памятью и диском. ARM в этой инструкции не проверялся; Linux lockfile заявлен для x86_64.

На своём компьютере (терминал или PowerShell):

```
ssh -p 22 root@185.11.247.226
```

При первом подключении сверяйте отпечаток SSH-ключа с администратором или консолью провайдера. Не подтверждайте неожиданную смену ключа без проверки причины. Пароли, приватные ключи и токены нельзя отправлять в чат или включать в эту инструкцию.

**Остальные команды выполняются на удалённом сервере**, если прямо не написано иначе. Не путайте сервер с терминалом ноутбука.

```
cat /etc/os-release
uname -m
free -h
df -h /
systemctl status ssh --no-pager
ss -lntp
```

Проверка исходящей сети:

```
getent ahostsv4 api.telegram.org
curl -4 -I --connect-timeout 15 --max-time 30 https://api.telegram.org
curl -4 -I --connect-timeout 15 --max-time 30 https://github.com
curl -4 -I --connect-timeout 15 --max-time 30 https://pypi.org
```

Ответ getent проверяет только DNS. Ping не проверяет TCP 443. Любой HTTP-ответ Telegram подтверждает установление HTTPS-соединения, но не действительность токена. Тайм-аут означает неуспешную проверку соединения.

Для установки дополнительно нужны реестр и адреса скачивания Docker Hub, files.pythonhosted.org и зеркала Ubuntu. Доступ к главной странице PyPI не гарантирует скачивание всех колёс; нужный заблокированный адрес определяйте по ошибке сборки.

Запрос администраторам при недоступности Telegram:

Для учебного бота на сервере 185.11.247.226 нужен исходящий HTTPS к api.telegram.org:443 либо согласованный HTTP/SOCKS5-прокси. Используется long polling, входящие подключения Telegram не требуются. Сейчас IPv4-соединение завершается тайм-аутом. Параметры доступа просим передать защищённым способом.

# 5. Установка Docker

Выберите один источник пакетов. Для проекта подходят Docker Engine и Compose из Ubuntu; этот вариант использован на текущем сервере. Не смешивайте docker.io и Docker CE без отдельного плана перехода.

Сначала проверьте:

```
docker --version
docker compose version
```

Если обе команды работают, не переустанавливайте Docker. Если он отсутствует:

```
apt-get update &&
apt-get install -y docker.io docker-compose-v2 git ca-certificates curl python3 &&
systemctl enable --now docker
```

Проверки:

```
docker version
docker compose version
docker run --rm hello-world
```

hello-world требует Docker Hub; ошибка загрузки может быть сетевой, а не ошибкой Docker. Для дополнительного файла нужен Compose 2.24.4+ с поддержкой !override. На текущем сервере версия 2.40.3 подходит. Используется команда `docker compose` с пробелом, а не старый отдельный docker-compose.

Альтернатива: Docker CE из официального репозитория, пакеты docker-ce, docker-ce-cli, containerd.io, docker-buildx-plugin, docker-compose-plugin. Следуйте актуальной инструкции https://docs.docker.com/engine/install/ubuntu/ с проверкой подписей. Переход действующего сервера на другой источник ради этого приложения не требуется.

# 6. Получение кода

На новом сервере:

```
git clone https://github.com/alexander1221/equipment_bot_cinema.git /opt/equipment_bot_cinema
cd /opt/equipment_bot_cinema
git branch --show-current
git log -1 --oneline
git status --short
```

Если каталог уже существует, изучите его; не удаляйте и не клонируйте поверх. Не путайте повторную установку с обновлением существующей базы.

Для воспроизводимого нового развёртывания можно закрепить проверенную версию приложения:

```
git checkout --detach 647ede50a04248c64c1b48a1e891f83a7b0309f2
```

Это необязательный шаг. В detached HEAD нет выбранной ветки; раздел обновления явно переключает чистое дерево на main. На существующем сервере сначала делайте копию и согласовывайте изменение версии.

Если репозиторий закрыт, владелец должен выдать доступ через deploy key или другой согласованный механизм. Не включайте GitHub-токен в URL, историю терминала или переписку.

# 7. Параметры и секреты

Рабочий каталог — /opt/equipment_bot_cinema.

```
cd /opt/equipment_bot_cinema
if [ ! -e .env ]; then
  install -m 600 .env.example .env
fi
chmod 600 .env
nano .env
```

На существующем сервере сохраняйте действующие значения. Для новой панели задайте разные длинные пароли вместо заглушек:

```
WEB_HOST=0.0.0.0
WEB_PORT=8080
WEB_ADMIN_PASSWORD=ЗАМЕНИТЕ_НА_ПАРОЛЬ_АДМИНИСТРАТОРА
WEB_SPECIALIST_PASSWORD=ЗАМЕНИТЕ_НА_ПАРОЛЬ_СПЕЦИАЛИСТА
TIMEZONE=Europe/Moscow
BOT_TOKEN=
TELEGRAM_PROXY=
```

Не оставляйте заглушки паролями. Для простоты используйте длинные значения из букв, цифр, дефиса и подчёркивания; #, $, пробелы и кавычки требуют корректного оформления dotenv/Compose. Не публикуйте вывод cat .env, docker compose config или docker inspect: там могут оказаться секреты.

В nano: Ctrl+O → Enter — сохранить; Ctrl+X — выйти. Не демонстрируйте экран при вводе секретов.

Создание ключа сессий без вывода и без замены существующего непустого ключа:

```
python3 - <<'PY'
from pathlib import Path
import secrets
p = Path('.env')
lines = p.read_text().splitlines()
present = any(x.startswith('WEB_SECRET_KEY=') and
              x.partition('=')[2].strip().strip('\"\'') for x in lines)
if not present:
    lines = [x for x in lines if not x.startswith('WEB_SECRET_KEY=')]
    lines.append('WEB_SECRET_KEY=' + secrets.token_urlsafe(48))
    p.write_text('\n'.join(lines) + '\n')
p.chmod(0o600)
print('Ключ сессий подготовлен; значение не выводится')
PY
```

| Параметр | Назначение |
|---|---|
| BOT_TOKEN | Токен от BotFather; не нужен в режиме web |
| WEB_ADMIN_PASSWORD | Пароль панели с доступом к сотрудникам |
| WEB_SPECIALIST_PASSWORD | Пароль специалиста по выдаче |
| WEB_SECRET_KEY | Стабильная подпись cookie; сохранить между перезапусками |
| ADMIN_IDS | Числовые Telegram ID через запятую, не @username |
| TELEGRAM_PROXY | Реальный HTTP/SOCKS5-прокси; пусто при прямом доступе |
| TELEGRAM_API_SERVER | Альтернативный Bot API; обычно пусто; не обычный прокси |
| REQUIRE_STUDENT_VERIFICATION | Оставить true: курс и профиль подтверждает сотрудник |

| Параметр | Назначение |
|---|---|
| MAX_BOOKING_DAYS | Обычный срок новой студенческой брони, 14 дней |
| MAX_ADVANCE_DAYS | Горизонт новой студенческой брони, 90 дней |
| PENDING_TTL_HOURS | Ожидание преподавателя, 48 часов |
| ISSUE_DEFAULT_TIME | По умолчанию 14:00 |
| ISSUE_PLACES | Места выдачи через точку с запятой |

DATABASE_URL задаёт основной Compose: sqlite+aiosqlite:////srv/data/equipment.db. Четыре слеша перед srv нужны для абсолютного пути. Не меняйте путь без переноса базы. WEB_HOST=0.0.0.0 относится к интерфейсам внутри контейнера; внешний доступ ограничивает публикация порта в Compose.

# 8. Запуск панели без Telegram

На новом сервере создайте дополнительный файл. Если он уже есть, сначала прочитайте и сохраните его, а не заменяйте вслепую.

```
cat > /opt/equipment_bot_cinema.web.yml <<'EOF'
services:
  equipment-bot:
    command: ["python", "main.py", "web"]
    ports: !override
      - "127.0.0.1:8080:8080"
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"
EOF
```

На текущем сервере подтверждён вариант без блока logging; добавить его можно при плановом пересоздании. Ограничение логов предотвращает неограниченный рост Docker-логов. !override заменяет исходный список портов целиком; не удаляйте тег. Без замены исходная общедоступная публикация 8080 может сохраниться.

Для новой копии создайте локальный .dockerignore через `nano .dockerignore` со следующим содержимым. Существующий файл дополняйте после проверки, не перезаписывайте:

```
.git
.env
.env.*
!.env.example
data/
.venv/
__pycache__/
.pytest_cache/
*.log
```

Dockerfile копирует только нужный код, а .dockerignore дополнительно исключает локальные данные и секреты из отправляемого контекста сборки. Локальные изменения .dockerignore не нужно автоматически коммитить в проект.

Запуск:

```
cd /opt/equipment_bot_cinema
docker compose -f docker-compose.yml \
  -f /opt/equipment_bot_cinema.web.yml up -d --build
```

Отдельный build не обязателен. setup.sh и установка Python-зависимостей на хост для Docker-режима не нужны. При первом старте создаются таблицы, преподаватели и начальный инвентарь.

Проверка:

```
docker compose -f docker-compose.yml \
  -f /opt/equipment_bot_cinema.web.yml ps
curl -sS -o /dev/null -w 'HTTP %{http_code}\n' http://127.0.0.1:8080/login
```

```
curl -I http://127.0.0.1:8080/inventory
```

Ожидаются Up, команда python main.py web, порт только 127.0.0.1:8080 и HTTP 200 на /login. Без авторизации /inventory должен перенаправлять на /login, обычно 303. Статус Up без запроса не подтверждает готовность.

Сразу после старта возможен единичный HTTP 000/Connection reset: приложение инициализируется. Повторите через несколько секунд. При устойчивой ошибке:

```
docker compose -f docker-compose.yml \
  -f /opt/equipment_bot_cinema.web.yml logs --tail=100 equipment-bot
```

# 9. HTTPS по IP через nginx

На действующем сервере этот раздел уже выполнен: сначала проверяйте, не выпускайте сертификат повторно без необходимости.

```
apt-get update && apt-get install -y nginx openssl
```

Если на 80/443 уже работает чужой сайт, согласуйте конфигурацию с администратором. Не останавливайте чужие службы. Порты смотрят через ss -lntp.

## 9.1. Первый выпуск временного сертификата

Самоподписанный сертификат шифрует трафик, но не имеет автоматически доверенного издателя. SAN должен содержать ваш реальный IP. Ключ остаётся закрытым. Следующая команда перезаписывает пару, поэтому повторный выпуск делайте только после сохранения старых файлов.

```
install -d -m 700 /etc/nginx/ssl/equipment-bot
(
  umask 077
  openssl req -x509 -nodes -newkey rsa:2048 -sha256 -days 90 \
    -keyout /etc/nginx/ssl/equipment-bot/server.key \
    -out /etc/nginx/ssl/equipment-bot/server.crt \
    -subj "/CN=185.11.247.226" \
    -addext "subjectAltName=IP:185.11.247.226"
)
```

Публичный отпечаток и даты:

```
openssl x509 -in /etc/nginx/ssl/equipment-bot/server.crt \
  -noout -dates -fingerprint -sha256
```

Передайте отпечаток доверенным способом. Перед исключением в браузере сверяйте адрес и SHA-256. Не приучайте пользователей игнорировать предупреждения без проверки. Приватный server.key не отправляется пользователям.

## 9.2. Сайт nginx

На новом сервере выполните блок целиком. Кавычки у EOF сохраняют переменные nginx буквально.

```
cat > /etc/nginx/sites-available/equipment-bot <<'EOF'
server {
    listen 80;
    server_name 185.11.247.226;
    return 301 https://185.11.247.226$request_uri;
}
server {
    listen 443 ssl;
    server_name 185.11.247.226;
    ssl_certificate /etc/nginx/ssl/equipment-bot/server.crt;
    ssl_certificate_key /etc/nginx/ssl/equipment-bot/server.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
```

```
}
EOF
ln -s /etc/nginx/sites-available/equipment-bot \
  /etc/nginx/sites-enabled/equipment-bot
nginx -t && systemctl enable --now nginx && systemctl reload nginx
```

Если ссылка уже есть, проверьте её назначение через ls -l; при правильной существующей ссылке повторный ln не нужен. Если nginx -t отказал, сначала исправьте конфигурацию. Не применяйте её принудительно.

### 9.3. Локальная проверка HTTPS

```
curl --cacert /etc/nginx/ssl/equipment-bot/server.crt \
  --resolve 185.11.247.226:443:127.0.0.1 \
  -sS -o /dev/null -w 'HTTPS %{http_code}\n' \
  https://185.11.247.226/login
```

Ожидается HTTPS 200. --resolve направляет запрос в локальный nginx, сохраняя IP сертификата. --cacert доверяет именно созданному сертификату и не отключает проверку. Локальный успех ещё не проверяет внешний маршрут.

## 10. Внешний доступ и файрвол

На своём компьютере откройте https://185.11.247.226, сверьте отпечаток и войдите с паролем администратора или специалиста из .env. Проверьте отклонение неправильного пароля, выход и требование повторного входа для закрытого раздела.

Для входящих соединений нужны TCP 443, TCP 80 для перенаправления и SSH-порт (обычно 22). Порт 8080 наружу не открывайте. В хостинге может быть отдельный сетевой экран поверх UFW.

```
ufw status verbose
ss -lntp
```

Если UFW уже активен и нужные правила отсутствуют, под контролем администратора:

```
ufw allow 22/tcp
ufw allow 80/tcp
ufw allow 443/tcp
```

Для изменённого SSH-порта разрешайте именно его. Не выполняйте ufw enable до проверки SSH-правил и доступности консоли провайдера: можно потерять доступ. Не отключайте файрвол целиком ради приложения.

Если локально HTTPS 200, а снаружи сайт не открывается, проверьте входящие правила, адрес, наличие слушателя 443 и внешний маршрут. Не обходите проблему общедоступным HTTP на 8080.

## 11. Полный режим и Telegram-прокси

### 11.1. Настройка токена и сети

Получите токен у @BotFather и введите в .env через nano непосредственно на сервере. Не отправляйте его в чат или curl-URL: это может попасть в историю и логи. Проверьте отсутствие второго работающего бота с тем же токеном на другом сервере или ноутбуке.

При прямом доступе TELEGRAM_PROXY оставляют пустым. При использовании согласованного прокси запишите один реальный URL. Примеры формата, не готовые рабочие адреса:

```
TELEGRAM_PROXY=socks5://LOGIN:PASSWORD@PROXY_HOST:PORT
```

или:

```
TELEGRAM_PROXY=http://PROXY_HOST:PORT
```

Администратор подтверждает тип, адрес, порт, способ авторизации и разрешение клиентского IP. Для HTTPS через HTTP-прокси обычно используется CONNECT, а схема URL самого прокси остаётся http://. MTProto-прокси из приложения Telegram для Bot API не подходит.

Спецсимволы в логине/пароле URL должны быть закодированы, а значение корректно записано в .env. Aiogram в этой реализации не подхватывает системные HTTP_PROXY/HTTPS_PROXY автоматически: нужен TELEGRAM_PROXY.

**127.0.0.1 в контейнере означает сам контейнер, а не сервер.** Пример socks5://127.0.0.1:10808 не заработает без реального прокси в том же контейнере. Для прокси на хосте администратор должен настроить доступный из Docker адрес и разрешения. Не публикуйте открытый прокси без авторизации. Для внешнего прокси проверьте доступность из Docker-сети, а не только с хоста.

## 11.2. Отдельный файл полного режима

Перед изменением сделайте копию по разделу 14. Создайте другой override-файл; web.yml оставьте для возврата к панели.

```
cat > /opt/equipment_bot_cinema.full.yml <<'EOF'
services:
  equipment-bot:
    command: ["python", "main.py", "run"]
    ports: !override
      - "127.0.0.1:8080:8080"
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"
EOF
cd /opt/equipment_bot_cinema
docker compose -f docker-compose.yml \
  -f /opt/equipment_bot_cinema.full.yml up -d --build
docker compose -f docker-compose.yml \
  -f /opt/equipment_bot_cinema.full.yml logs --tail=100 equipment-bot
```

Это тот же Compose-проект и сервис: заменяется контейнер, каталог data остаётся. Передавайте только один из дополнительных файлов: web или full. Не меняйте имя проекта и не запускайте вторую копию polling.

Бот на старте выполняет getMe и задаёт команды. Если токен или сеть не работают, процесс может завершаться, и панель тоже не поднимется. Для временного возврата:

```
docker compose -f docker-compose.yml \
  -f /opt/equipment_bot_cinema.web.yml up -d
```

После успешного старта повторите HTTP/HTTPS-проверки и отправьте /start настоящему боту. Его ответ подтверждает фактическую работу токена и сети. Не исправляйте сетевые ошибки отключением TLS-проверки.

# 12. Первичная настройка пользователей

1. Войти в панель с паролем администратора; открыть «Сотрудники». 2. Уточнить ФИО преподавателей и специалиста. Начальные записи уже создаются автоматически, но Telegram-аккаунты нужно связать. 3. Безопасно передать каждому сотруднику его персональное приглашение. Сотрудник открывает ссылку своим аккаунтом Telegram и нажимает Start. Не публикуйте приглашения в общей группе: они связывают аккаунт с ролью. 4. Проверить, что специалист получает карточки новых студентов. Пока он не подключён, профили видны в панели. 5. Студент заполняет курс, ФИО и группу. Сотрудник сверяет данные и подтверждает профиль. Не отключайте REQUIRE_STUDENT_VERIFICATION ради демонстрации. 6. Сверить начальный инвентарь и количества с фактическим оборудованием до реальной работы.

Контрольный сценарий полного режима на согласованной тестовой заявке:

• Студент выбирает свободные даты и оборудование, отправляет заявку; преподаватель получает карточку.

• Преподаватель подтверждает или отклоняет сразу либо выбирает «с комментарием». Комментарий до 500 символов сохраняется и отправляется студенту. Ввод можно отменить или завершить «Без комментария».

• Специалист назначает выдачу, отправляет студенту время и место; при фактической выдаче отмечает «Выдано».

• После физического возврата и проверки комплектности сотрудник отмечает «Возвращено». Заявка перестаёт блокировать оборудование.

Досрочный возврат: кнопка студента «Хочу вернуть раньше» только уведомляет специалиста. Студент сам не закрывает выданную заявку. Просрочка также не равна возврату: нужна отметка сотрудника.

Продление: карточка заявки → «Изменить даты или состав» → изменить «По» → сохранить. Конфликтующие брони не дадут продлить. У выданной заявки нельзя менять дату начала. Обычный 14-дневный лимит новой студенческой брони при редактировании сотрудником не применяется. Сам студент срок не продлевает. В web-only Telegram-уведомления отсутствуют.

# 13. Управление приложением и перезагрузка

Все команды ниже выполняются из /opt/equipment_bot_cinema и приведены для web-only. После перехода на полный режим замените .web.yml на .full.yml во всех командах управления.

Состояние и логи:

```
cd /opt/equipment_bot_cinema
docker compose -f docker-compose.yml \
  -f /opt/equipment_bot_cinema.web.yml ps -a
docker compose -f docker-compose.yml \
  -f /opt/equipment_bot_cinema.web.yml logs --tail=100 equipment-bot
```

Остановка — отдельное действие:

```
docker compose -f docker-compose.yml \
  -f /opt/equipment_bot_cinema.web.yml stop
```

Включение существующего остановленного контейнера:

```
docker compose -f docker-compose.yml \
  -f /opt/equipment_bot_cinema.web.yml start
```

Не выполняйте stop и start подряд, если хотите оставить контейнер выключенным. Во время остановки 502 от nginx ожидаем; после старта nginx сам снова начнёт отдавать панель, перезапускать его не нужно.

**Новые значения .env не применяются через start/restart.** Для их применения пересоздайте контейнер:

```
docker compose -f docker-compose.yml \
  -f /opt/equipment_bot_cinema.web.yml up -d --force-recreate
```

Изменения кода требуют сборки с --build. Если контейнера ещё нет или его удалили, вместо start нужен up -d с правильными файлами. Обычный docker compose up без override не используйте.

Проверки служб:

```
systemctl is-enabled docker nginx
systemctl status nginx --no-pager
nginx -t && systemctl reload nginx
```

Docker и nginx должны быть enabled. Политика restart: unless-stopped поднимает контейнер после перезагрузки, если он не был остановлен вручную. После ручного stop он останется остановленным даже при перезагрузке; нужен start. Выход из SSH не останавливает приложение, запущенное с -d.

nginx reload требуется после изменения nginx-конфигурации или сертификата, а не после обычного старта приложения. Перезапуск процесса сбрасывает незавершённые диалоги бота, но не уже отправленные заявки в базе.

# 14. Резервное копирование

Создавайте копию до обновлений, смены режима, переноса и регулярно в эксплуатации. Образ Docker не содержит актуальную базу. Не копируйте только equipment.db во время записи обычным cp: SQLite может хранить часть изменений в WAL. Используйте штатный SQLite backup API.

В комплекте находится equipment-backup.py. Он делает согласованный снимок базы, проверяет integrity_check, сохраняет .env, Compose, override-файлы, IP-конфигурацию nginx, сертификат с ключом и SHA коммита, затем создаёт tar.gz. Код скрипта доступен для проверки. Он рассчитан на пути из паспорта; на другом развёртывании их нужно изменить.

Передача скрипта с компьютера сопровождающего (из папки распакованного комплекта):

```
scp -P 22 equipment-backup.py root@185.11.247.226:/root/equipment-backup.py
```

На сервере:

```
install -m 600 /root/equipment-backup.py /usr/local/sbin/equipment-backup.py
/usr/bin/python3 /usr/local/sbin/equipment-backup.py
```

Результат должен сообщить integrity_check=ok и путь к архиву в /var/backups/equipment-bot. Снимок можно делать при работающем приложении. Имена используют UTC, независимо от часового пояса приложения.

Запишите рядом с копией активный режим web/full и дату проверки. Если позже добавлен доменный сайт или Certbot, дополните состав копии: скрипт знает только перечисленные в нём пути и IP-сертификат. Дополнительные сайты и /etc/letsencrypt автоматически не сохраняются.

Архив включает секреты, приватный TLS-ключ и персональные данные. Права доступа ограничены root, но tar.gz сам по себе **не зашифрован**. Храните и передавайте его через защищённый канал; не публикуйте в GitHub, каталоге nginx или чате.

На компьютере администратора, заменив имя на реальный архив:

```
scp -P 22 root@185.11.247.226:/var/backups/equipment-bot/ИМЯ_АРХИВА.tar.gz .
```

Копии на том же диске не защищают от потери сервера. Вынесите их на отдельное закрытое хранилище, желательно с шифрованием. Владелец назначает ответственного, срок хранения и проверку восстановления.

# 15. Ежедневные копии по расписанию

Сначала вручную проверьте скрипт и целостность копии. Затем согласуйте время и установите cron, если его нет:

```
apt-get install -y cron
systemctl enable --now cron
timedatectl
```

Пример запуска ежедневно в 02:15 **по системному часовому поясу сервера**. TIMEZONE=Europe/Moscow в приложении не меняет часовой пояс cron. Проверьте timedatectl. На новом развёртывании:

```
cat > /etc/cron.d/equipment-bot-backup <<'EOF'
15 2 * * * root /usr/bin/python3 /usr/local/sbin/equipment-backup.py >> /var/log/equipment-backup.log
2>&1
EOF
chmod 644 /etc/cron.d/equipment-bot-backup
```

Если такой файл уже существует, сначала изучите его. После первого планового запуска проверьте:

```
ls -lt /var/backups/equipment-bot
tail -n 30 /var/log/equipment-backup.log
df -h /
```

Скрипт оставляет каталог снимка и архив и не удаляет старые копии. Следите за местом. Срок хранения и очистку подтверждённых старых копий согласует владелец; не добавляйте слепое удаление по маске. Автоматический вынос с сервера и оповещение об ошибках требуют отдельной

настройки по правилам организации. Сам cron не доказывает защищённость: нужны свежие копии, контроль ошибок и пробное восстановление.

# 16. Восстановление базы

Восстановление заменяет текущие данные. Нужны разрешение владельца, окно простоя, копия текущего состояния и выбранная проверенная резервная копия. Предпочтительно сначала проверить восстановление на отдельной машине без действующего BOT_TOKEN.

Если сохранился только архив, сначала просмотрите его состав через tar -tzf. Распаковывайте доверенный архив в отдельный закрытый каталог, не поверх приложения. Проверьте дату и commit.txt. Далее пример использует уже распакованный каталог в /var/backups/equipment-bot/ВЫБРАННАЯ_ДАТА.

Остановите приложение и убедитесь по ps -a, что оно остановлено (для полного режима используйте full.yml):

```
cd /opt/equipment_bot_cinema
docker compose -f docker-compose.yml \
  -f /opt/equipment_bot_cinema.web.yml stop
```

Замените путь к выбранному снимку в скрипте ниже. Скрипт проверяет копию, сохраняет старый каталог data целиком и помещает восстановленную базу в новый. Работает только при остановленном приложении.

```
python3 - <<'PY'
from pathlib import Path
from datetime import datetime, timezone
import os, shutil, sqlite3
os.umask(0o077)
source = Path('/var/backups/equipment-bot/ВЫБРАННАЯ_ДАТА/equipment.db')
if not source.is_file():
    raise SystemExit('Укажите существующий файл резервной копии')
with sqlite3.connect(source.as_uri() + '?mode=ro', uri=True) as conn:
    if conn.execute('PRAGMA integrity_check').fetchone()[0] != 'ok':
        raise SystemExit('Копия не прошла проверку')
data = Path('/opt/equipment_bot_cinema/data')
stamp = datetime.now(timezone.utc).strftime('%Y%m%dT%H%M%SZ')
saved = data.with_name('data.before-restore-' + stamp)
if data.exists():
    data.rename(saved)
data.mkdir(mode=0o700)
shutil.copyfile(source, data / 'equipment.db')
print('Прежние данные сохранены:', saved)
print('База восстановлена; запустите и проверьте приложение')
PY
```

Не подмешивайте старые WAL/SHM к восстановленному снимку. Права рассчитаны на текущий образ, запускающий процесс root в контейнере; если его пользователь изменится, согласуйте права с ним.

При утрате .env восстановите app.env отдельно с правами 600. Не заменяйте актуальные токены и пароли старыми автоматически. nginx и сертификаты восстанавливают только при необходимости, затем проверяют nginx -t. Одна замена базы не требует нового сертификата.

Запустите нужный режим через up -d, после замены .env используйте --force-recreate. Проверьте HTTP/HTTPS, вход, инвентарь, сотрудников, студентов и заявки. В полном режиме проверьте ответ бота и уведомления. Старый data сохраняйте до подтверждения владельцем.

# 17. Обновление и откат

Не обновляйте в момент выдачи. Перед изменением создайте резервную копию базы и настроек, запишите SHA и режим, изучите изменения новой версии.

```
cd /opt/equipment_bot_cinema
git status --short
git rev-parse HEAD
docker compose -f docker-compose.yml \
  -f /opt/equipment_bot_cinema.web.yml ps
```

При изменениях кода или неизвестных файлах сначала разберитесь с ними. .dockerignore из этой инструкции может быть ожидаемым локальным файлом. Не используйте reset --hard или git clean для устранения затруднения: они могут уничтожить работу. .env и data должны оставаться локальными.

Для чистой копии:

```
git fetch origin
git log --oneline HEAD..origin/main
git switch main
git merge --ff-only origin/main
```

Если switch или ff-only отказал, остановитесь и обратитесь к сопровождающему; не применяйте force. Сначала успешно соберите новую версию, затем пересоздайте сервис. В полном режиме замените web.yml на full.yml.

```
docker compose -f docker-compose.yml \
  -f /opt/equipment_bot_cinema.web.yml build
docker compose -f docker-compose.yml \
  -f /opt/equipment_bot_cinema.web.yml up -d
docker compose -f docker-compose.yml \
  -f /opt/equipment_bot_cinema.web.yml logs --tail=100 equipment-bot
curl -sS -o /dev/null -w 'HTTP %{http_code}\n' http://127.0.0.1:8080/login
```

Успешная сборка не заменяет тесты. Dockerfile не копирует tests в production-образ; не заявляйте, что pytest внутри него проверил проект. Изменения тестируются разработчиком/CI или отдельной тестовой средой.

В текущем коде create_all дополняется добавлением отдельных известных столбцов, но это не универсальная система миграций. Новая версия может требовать отдельной миграции. Читайте примечания к обновлению.

Для отката совместимой версии на чистом дереве используйте записанный прежний SHA:

```
git switch --detach СТАРЫЙ_SHA
docker compose -f docker-compose.yml \
  -f /opt/equipment_bot_cinema.web.yml up -d --build
```

Если менялась схема базы, одного отката кода может быть недостаточно: согласуйте восстановление БД. Оно теряет изменения после снимка; решение принимает владелец. Не подставляйте исторический SHA 647ede5 автоматически вместо реально записанной последней рабочей версии.

# 18. Домен и обслуживание сертификатов

## Временный сертификат

Проверяйте реальную дату истечения:

```
openssl x509 -in /etc/nginx/ssl/equipment-bot/server.crt -noout -dates
```

Самоподписанный сертификат автоматически не продлевается. До истечения сохраните текущие server.crt и server.key в закрытом каталоге, выпустите новую пару по разделу 9, выполните nginx -t и reload, проверьте HTTPS. При сбое восстановите сохранённую пару. Новый сертификат имеет другой отпечаток, его нужно передать пользователям заново безопасным способом.

Доверенные IP-сертификаты зависят от поддержки центра сертификации и клиента. В этом развёртывании такой сертификат не настроен; не обещайте отсутствие предупреждений для текущего сертификата.

## Домен с доверенным сертификатом

Владелец назначает домен. Создайте DNS A-запись на сервер. AAAA добавляйте только при реально работающем IPv6. Проверьте DNS с внешнего компьютера и доступность 80/443.

Сохраните конфигурацию nginx. Создайте отдельный файл /etc/nginx/sites-available/equipment-domain через nano. В примере equipment.example.org —

заглушка; замените на настоящий домен.

```
server {
    listen 80;
    server_name equipment.example.org;
    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Включите новый сайт и получите сертификат (в certbot тоже нужен реальный домен):

```
ln -s /etc/nginx/sites-available/equipment-domain \
  /etc/nginx/sites-enabled/equipment-domain
nginx -t && systemctl reload nginx
apt-get update && apt-get install -y certbot python3-certbot-nginx
certbot --nginx -d equipment.example.org --redirect
```

Введите действующий email владельца и прочитайте условия центра. Certbot настроит HTTPS. Проверьте:

```
nginx -t
certbot certificates
certbot renew --dry-run
systemctl list-timers --all
```

Продление считается настроенным после успешного dry-run и проверки расписания. Откройте доменный сайт с компьютера без предупреждений. При необходимости добавьте PUBLIC_BASE_URL=https://ВАШ_ДОМЕН в .env и пересоздайте контейнер. Удаляйте старый IP-сайт только после проверки домена и согласования с владельцем. Дополните резервное копирование доменной конфигурацией и /etc/letsencrypt.

# 19. Диагностика неисправностей

## SSH не подключается

Проверьте адрес, пользователя, порт, консоль провайдера, systemctl status ssh и ss -lntp. Connection refused описывает выбранное соединение, а не доказывает отсутствие SSH вообще. Не отключайте проверку ключей и не публикуйте пароль.

## Ошибка !override

Проверьте docker compose version: нужен современный Compose 2.24.4+. Не удаляйте тег ради запуска — иначе можно открыть 8080 наружу. Обновите пакет выбранного источника или согласуйте другую конфигурацию.

## Предупреждение про Bake/buildx

Если сборка завершилась FINISHED и HTTP-проверка прошла, это предупреждение не мешает работе. Не меняйте источник всех Docker-пакетов ради него. При желании администратор может установить совместимый buildx из выбранного источника.

## Не скачивается образ или пакет

Различайте DNS, тайм-аут, запрет сети, TLS, ошибку хэша и нехватку ресурсов. Проверяйте адрес из ошибки, free -h, df -h /, docker system df. Не отключайте проверки TLS/хэшей/подписей и не заменяйте lockfile непроверенными версиями. Не выполняйте docker system prune -a или volume prune без инвентаризации: это может затронуть другие сервисы.

## HTTP 000 или Connection reset

Единичный сбой сразу после старта может быть инициализацией. Повторите запрос, затем проверьте ps -a и logs. Постоянный HTTP 000 — не успешная проверка готовности.

## nginx 502

Проверьте контейнер и http://127.0.0.1:8080/login. После ручного stop это ожидаемо. Исправление сертификата не запустит остановленное приложение.

## Локально HTTPS 200, снаружи не работает

Проверьте входящие правила хостинга/UFW, IP/домен и слушатель 443. Локальный --resolve обходит внешний маршрут и не проверяет его. Не открывайте 8080 наружу как обход nginx.

## Сертификат не доверен, просрочен или не совпадает имя

Для самоподписанного издателя предупреждение ожидаемо; проверяйте адрес и отпечаток. Для срока обновите сертификат; для имени проверьте SAN. Не используйте curl -k вместо устранения причины.

## Изменённый пароль не работает

Проверьте нужный .env, формат и опечатки без вывода значений. Выполните up -d --force-recreate: start/restart не перечитывают env_file в существующем контейнере. Для подписи cookie нужен постоянный WEB_SECRET_KEY; смена ключа сбрасывает сессии.

## Бот молчит, панель работает

Проверьте команду в ps: python main.py web не запускает Telegram. В web-only нет уведомлений и фонового цикла. Для полного режима нужны full.yml, BOT_TOKEN и работающая сеть.

## Telegram отклонил токен

Проверьте или перевыпустите токен через BotFather безопасно. Убедитесь, что .env применён. Не отправляйте URL с токеном в отчёт.

## TelegramNetworkError / ProxyError

Проверьте связь, реальный адрес/тип прокси, разрешение сервера и доступ из контейнера. 127.0.0.1 не является адресом хоста; MTProto не подходит. Пока администраторы решают вопрос, можно вернуть web-only через up -d с web.yml. Не отключайте TLS.

## TelegramConflictError / второй getUpdates

Один токен используется несколькими процессами. Найдите второй контейнер, сервер или ноутбук и согласованно остановите лишний. Не создавайте ещё одну polling-копию для диагностики.

## Сотрудник не получает заявки

Проверьте полный режим, персональную привязку аккаунта и то, что пользователь не блокировал бота. Пока преподаватель не подключён, сотрудник может согласовать в панели. Регистрации студентов должен подтверждать специалист.

## После stop и перезагрузки контейнер не включился

Это поведение unless-stopped: выполните start. Если контейнер удалён — up -d. Для nginx и Docker должен быть enabled.

### Что отправлять сопровождающему

Время и симптом, SHA и режим, ps -a, относящиеся к ошибке строки logs, HTTP-код, nginx -t. Удаляйте пароли, токены, прокси-URL с авторизацией, приглашения и персональные данные. Не отправляйте .env, приватные ключи, полный inspect или compose config.

## 20. Приёмка и передача сопровождения

### Проверки панели

• Зафиксированы сервер, SSH-порт, пути, SHA и режим.

• Docker/nginx включены в автозапуск; понятна политика unless-stopped.

• Порт 8080 доступен только через 127.0.0.1.

• HTTP /login и HTTPS с проверкой сертификата возвращают 200.

• Внешний браузер открывает сайт; вход, неверный пароль, выход и роли проверены.

• Есть срок сертификата, ответственный за замену и способ проверки отпечатка.

• .env имеет права 600, секреты не находятся в Git и документах.

• stop/start проверены в согласованное окно; nginx не требует перезапуска.

### Проверки бота — только после его фактического включения

• Бот отвечает на /start, активен только один polling-экземпляр.

• Сотрудники подключили свои аккаунты, специалист видит регистрации.

• Проверены согласование с/без комментария, выдача и возврат.

• Подтверждены уведомления; учтены фоновые задачи только полного режима.

### Проверки сопровождения

• Создан согласованный SQLite-снимок с integrity_check=ok.

• Есть закрытая копия .env, Compose, nginx и TLS.

• Копии вынесены с сервера; на отдельной среде проверено восстановление.

• Назначены расписание, контроль ошибок, срок хранения и ответственный.

• Записан порядок обновления/отката и риск восстановления старой базы.

• Владелец знает, кто сопровождает сервер, Telegram, домен и резервные копии.

Шаблон передачи, без значений секретов:

```
Дата и сопровождающий:
Сервер / ОС / SSH-порт:
Проект / commit:
Режим web или full:
URL панели:
Где безопасно хранятся доступы:
Срок сертификата и ответственный:
Последняя копия и проверка целостности:
Место внешней копии:
Дата и место пробного восстановления:
Статус доступа Telegram / прокси:
Оставшиеся ограничения:
Контакты владельца и администратора:
```

При смене сопровождающего владелец пересматривает доступы. Не передавайте общий root-пароль внутри инструкции; выдавайте индивидуальный доступ по правилам организации.

## Шпаргалка путей

| Путь | Назначение |
|---|---|
| /opt/equipment_bot_cinema | Код и основной Compose |
| /opt/equipment_bot_cinema/.env | Секреты и параметры |
| /opt/equipment_bot_cinema/data | Постоянная SQLite-база и WAL/SHM |
| /opt/equipment_bot_cinema.web.yml | Панель без Telegram |
| /opt/equipment_bot_cinema.full.yml | Полный режим, если создан |
| /etc/nginx/sites-available/equipment-bot | Сайт по IP |
| /etc/nginx/sites-enabled/equipment-bot | Ссылка на активный сайт |
| /etc/nginx/ssl/equipment-bot | Сертификат и закрытый ключ |
| /var/log/nginx | Логи nginx |
| /usr/local/sbin/equipment-backup.py | Установленный скрипт копирования |
| /var/backups/equipment-bot | Локальные копии после настройки |

Частые команды из каталога проекта:

```
# Запустить ранее созданную панель
 docker compose -f docker-compose.yml -f /opt/equipment_bot_cinema.web.yml start
# Остановить панель
 docker compose -f docker-compose.yml -f /opt/equipment_bot_cinema.web.yml stop
# Состояние, включая остановленные контейнеры
 docker compose -f docker-compose.yml -f /opt/equipment_bot_cinema.web.yml ps -a
# Проверить приложение
 curl -sS -o /dev/null -w 'HTTP %{http_code}\n' http://127.0.0.1:8080/login
```

Не удаляйте data, не запускайте повторную инициализацию «для очистки ошибки», не используйте down/удаление ресурсов как первый способ диагностики. Готовность подтверждается функциональными запросами и проверкой данных, а не только наличием контейнера.